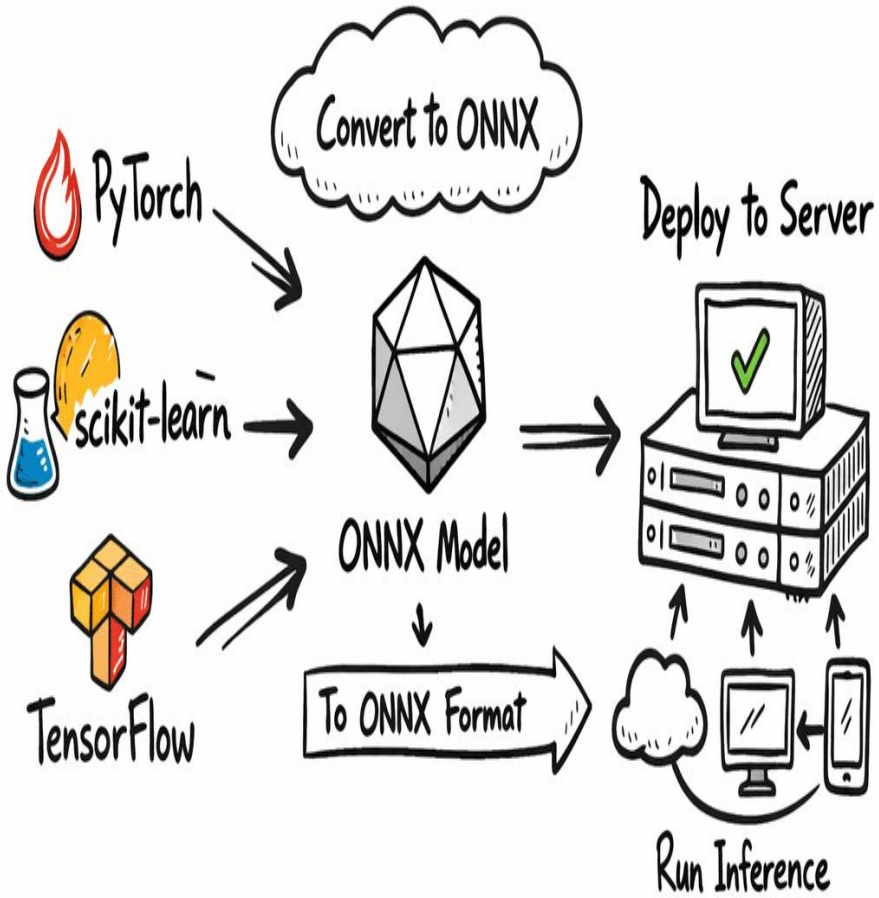
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN – ĐHQG TP.HCM

Học phần: AIoT – Trí tuệ nhân tạo vạn vật kết nối

Chuẩn ONNX và tối ưu hóa đồ thị trong ONNX Runtime

Tối ưu hóa suy luận biên
Thực nghiệm trên mô hình phân loại Dodo (ResNet50)



NHÓM 06 - Data4Sense

Trần Ngọc Phương Dung – 25C01028
Cao Anh Tuấn – 25C01025
Nguyễn Duy Thanh – 25C01047
Mai Nguyễn Hữu Hùng – 25C01032
Nguyễn Hồ Trường Sơn – 25C01020

Giảng viên: TS. Lê Duy Tân

TP. Hồ Chí Minh, tháng 10/2026

GitHub repo: **github.com/sonnguyen112/AIoT**

Nội dung trình bày

01 **Bối cảnh & vấn đề**
Rào cản đưa mô hình AI xuống thiết bị AIoT

02 **Tổng quan ONNX**
Nguồn gốc, khái niệm, cấu trúc tệp .onnx

03 **Tách rời huấn luyện & suy luận**
Lợi ích và điều kiện áp dụng

04 **ONNX Runtime**
Kiến trúc, phân mảnh đồ thị, Execution Provider

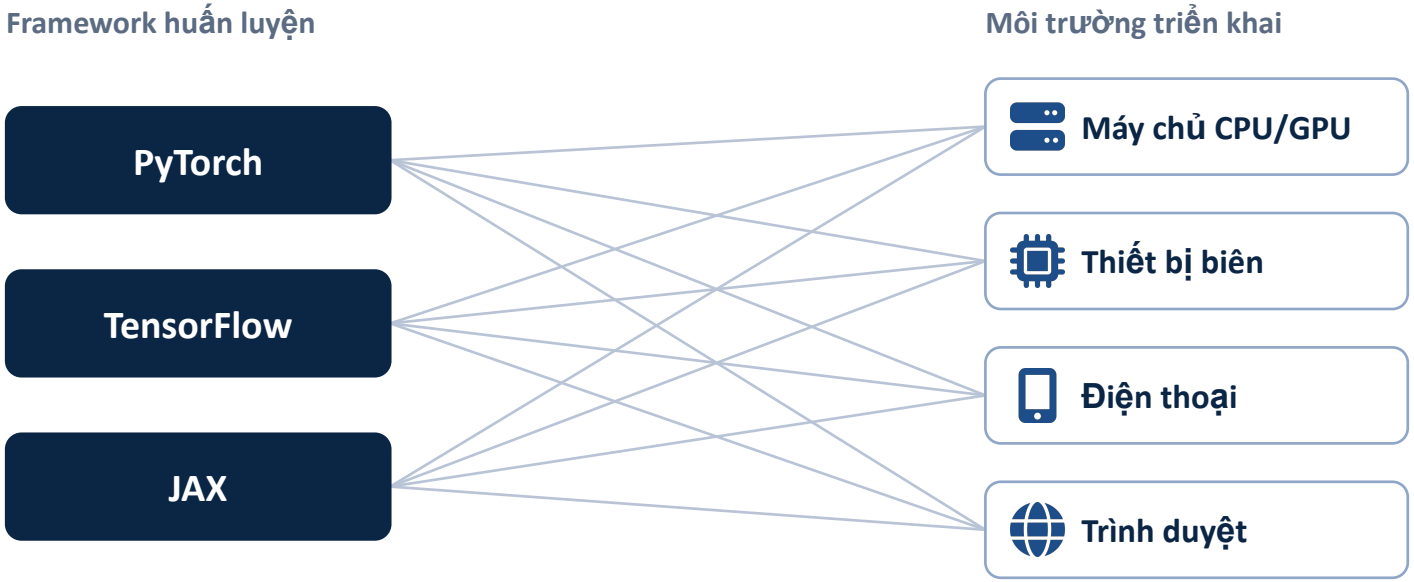
05 **Tối ưu hóa đồ thị**
Constant Folding, Node Fusion, Layout

06 **Bộ nhớ đệm**
Memory Arena, I/O Binding

07 **Thực nghiệm mô hình Dodo**
Chuyển đổi ResNet50, benchmark và kết quả

08 **Kết luận**

Rào cản khi đưa mô hình AI xuống thiết bị AIoT



Mỗi cặp framework – phần cứng cần một đường tích hợp riêng; hệ thống sản xuất dễ bị buộc phải kéo theo cả framework huấn luyện.

Thiết bị biên: tài nguyên hạn chế

Pin & năng lượng
Suy luận liên tục gây hao pin, quá nhiệt

RAM & bộ nhớ Flash
Không đủ chỗ cho framework nặng hàng GB

Năng lực tính toán
Chip nhỏ nhưng phải đáp ứng độ trễ thấp

Giải pháp: **ONNX** – chuẩn mở trao đổi mô hình + **ONNX Runtime** – engine suy luận hiệu năng cao, đa nền tảng

ONNX là gì? Nguồn gốc và khái niệm

Open Neural Network Exchange (ONNX)

Định dạng mở biểu diễn mô hình học máy dưới dạng đồ thị luồng dữ liệu, đóng vai trò biểu diễn trung gian (IR) giữa framework huấn luyện và công cụ thực thi.

ONNX không phải: framework huấn luyện | engine tính toán | ứng dụng hoàn chỉnh

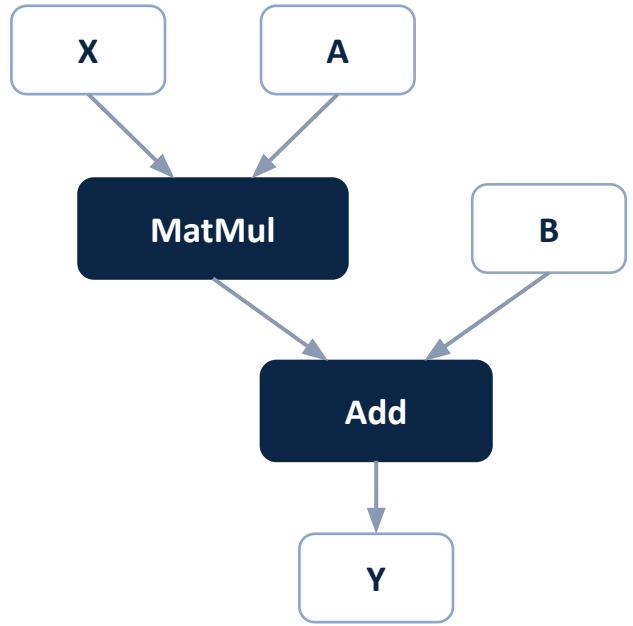
Toffee
Dự án nội bộ của nhóm PyTorch tại Facebook

09/2017
Facebook & Microsoft đổi tên, công bố ONNX

2019
Chuyển giao Linux Foundation (LF AI) để bảo đảm tính trung lập

Hiện nay
Chuẩn mở công nghiệp

Ví dụ: đồ thị của Y = X·A + B



Cạnh mang tensor · mỗi nút là một toán tử

Bên trong tệp .onnx: cấu trúc Protobuf phân cấp

ModelProto vỏ bọc + siêu dữ liệu định danh mô hình

`ir_version` `opset_import` `producer_name` `metadata_props`

GraphProto đồ thị có hướng không chu trình (DAG)

ValueInfoProto
Cổng vào/ra: tên, kiểu, shape
`float32[N,3,224,224]`

NodeProto
Toán tử: Conv, MatMul, Relu…
`op_type = "Conv"`

TensorProto
Initializer: trọng số, bias
`W: float32[64,3,7,7]`

Ví dụ lấy từ mô hình ResNet50 của nhóm (N = batch động)

Protocol Buffers (Google)

Tuần tự hóa nhị phân nhỏ gọn, đọc được trên nhiều nền tảng và ngôn ngữ

Opset

Phiên bản tập toán tử (miền ai.onnx); runtime đích phải hỗ trợ

Tách logic và tham số

Nút mô tả phép tính, initializer giữ trọng số → mô tả được từ bộ phân loại nhỏ đến mô hình hàng tỷ tham số

Giới hạn 2 GB / thông điệp Protobuf

Giải pháp external data: cấu trúc đồ thị giữ trong .onnx, các ma trận trọng số lớn được lưu ở tệp nhị phân riêng bên ngoài.

Tách rời huấn luyện và suy luận



Khi xuất (tracing), điều gì bị loại bỏ?

- Mã Python và các vòng lặp không tạo ra nút tính toán
- Cơ chế theo dõi gradient phục vụ lan truyền ngược
- Overhead của trình thông dịch Python và GIL
- Phụ thuộc hệ sinh thái huấn luyện nặng hàng GB

Kết quả

- Đồ thị tĩnh "đóng băng": chỉ còn các phép toán trên tensor
- Dịch vụ suy luận gọn cỡ vài MB thay vì hàng GB
- MLOps: nghiên cứu tự do → chuẩn hóa → triển khai mọi nơi

Lợi ích và điều kiện áp dụng ONNX

Khả năng tương tác

Nghiên cứu bằng PyTorch, triển khai bằng C++, C#, Java, Python…

Tính di động

Một biểu diễn thống nhất cho CPU, GPU, máy chủ, thiết bị biên và bộ tăng tốc

Tối ưu chuyên biệt

Nhà cung cấp runtime/phần cứng tối ưu ONNX thay vì phải hỗ trợ cả framework

Giảm độ phức tạp triển khai

Dễ đóng gói, kiểm thử và quản lý phiên bản môi trường suy luận

Chuẩn hóa quy trình

Mô hình từ nhiều framework đi chung một đường ống kiểm thử và triển khai

Điều kiện & giới hạn

- Phụ thuộc opset, kiểu dữ liệu, toán tử và Execution Provider
- Toán tử tùy biến, luồng điều khiển động phức tạp → phân rã, custom op hoặc sửa kiến trúc
- Hình dạng động phải khai báo khi xuất; nếu cố định shape, mô hình không nhận kích thước khác
- Luôn kiểm chứng: onnx.checker + so sánh đầu ra PyTorch và ORT trên cùng dữ liệu

Ví dụ kiểm chứng (Python)

```
onnx.checker.check_model(model)
np.testing.assert_allclose(
    torch_out, ort_out,
    rtol=1e-3, atol=1e-5)
```

ONNX Runtime: kiến trúc và Execution Provider

ONNX là bản thiết kế tĩnh · **ONNX Runtime (ORT)** là cỗ máy thực thi hướng phần cứng · **Execution Provider (EP)** là lớp giao diện đóng gói thư viện tăng tốc của từng loại phần cứng

1 Nạp mô hình

Phân tích tệp Protobuf thành đồ thị trong bộ nhớ

2 Tối ưu độc lập phần cứng

Đơn giản hóa đồ thị trước khi phân mảnh

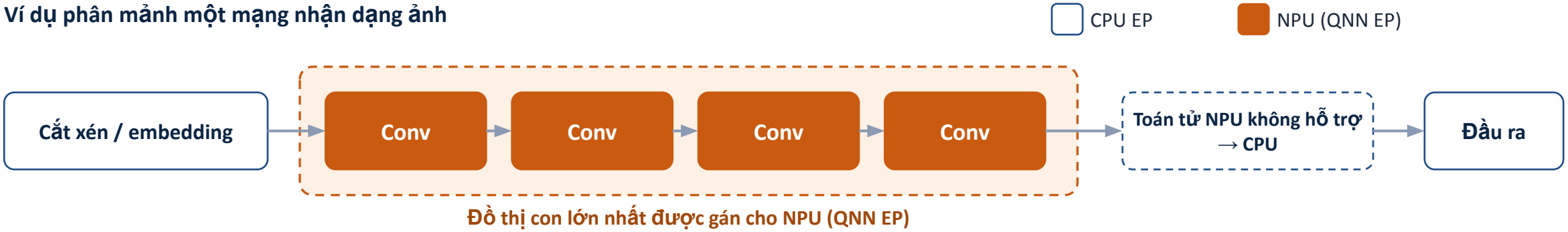
3 Phân mảnh đồ thị

GetCapability() từng EP theo thứ tự ưu tiên; gán tham lam đồ thị con lớn nhất

4 Thực thi

Mỗi đồ thị con chạy trên EP được gán; CPU EP là điểm rơi cuối cùng

Ví dụ phân mảnh một mạng nhận dạng ảnh



Hệ sinh thái Execution Provider trong AIoT và đám mây

Execution Provider	Phần cứng tiêu biểu	Đặc trưng & mục tiêu tối ưu
CPU (MLAS)	CPU x86, ARM	Tương thích toàn diện; tận dụng AVX/NEON và đa luồng; phương án dự phòng (fallback) bắt buộc
CUDA / TensorRT	GPU NVIDIA (Data Center, Jetson)	Tối ưu cho Tensor Cores; TensorRT nổi trội với lượng tử hóa INT8 và hợp nhất lớp
OpenVINO	CPU, iGPU, NPU của Intel	Phân bổ tải không đồng nhất (AUTO mode); khai thác NPU Intel mà không cần viết lại mã
CoreML	Apple Silicon (M-series, A-series)	Đẩy toán tử lên Apple Neural Engine, tiết kiệm năng lượng trên iPhone/Mac
QNN	Qualcomm Snapdragon (Hexagon NPU)	Giảm độ trễ và điện năng trên Android/Windows ARM; mô hình QDQ, dịch sang lệnh DSP/NPU
NNAPI	Android CPU, GPU, NPU	Lớp trừu tượng chuẩn, tự tìm bộ tăng tốc khả dụng trên thiết bị Android

Tối ưu hóa đồ thị: nguyên tắc và 3 mức tối ưu

Nguyên tắc

- Biến đổi đồ thị nhưng vẫn bảo toàn ngữ nghĩa của mô hình.
- Giảm số operator, lần gọi kernel và tensor trung gian.
- Giảm chi phí tính toán và truy cập bộ nhớ khi suy luận.

Hai chế độ áp dụng

Online: Tối ưu khi khởi tạo InferenceSession; thuận tiện nhưng làm tăng thời gian khởi tạo.

Offline: Tối ưu trước và lưu graph đã tối ưu; giúp giảm công việc khi khởi tạo session.

Các tối ưu phụ thuộc Execution Provider hoặc phần cứng cần được sử dụng trong môi trường tương thích.

Các mức tối ưu có tính tích lũy — mức sau bao gồm các tối ưu của mức trước

ORT_ENABLE_BASIC

- Chạy trước Graph Partitioning.
- Constant Folding và loại bỏ node dư thừa.
- Fusion cơ bản: Conv–Add, Conv–Mul, Conv–BatchNorm.
- Có thể áp dụng độc lập với Execution Provider.

Constant Folding → Node Elimination → Basic Fusion

ORT_ENABLE_EXTENDED

↓ bao gồm Basic

- Bao gồm toàn bộ Basic Optimization.
- Extended Optimization chạy sau Graph Partitioning.
- Gộp các chuỗi operator phù hợp thành fused operator.
- Khả năng áp dụng phụ thuộc từng Execution Provider.

Ví dụ trên CPU: Conv → ReLU thành Fused Conv–Activation

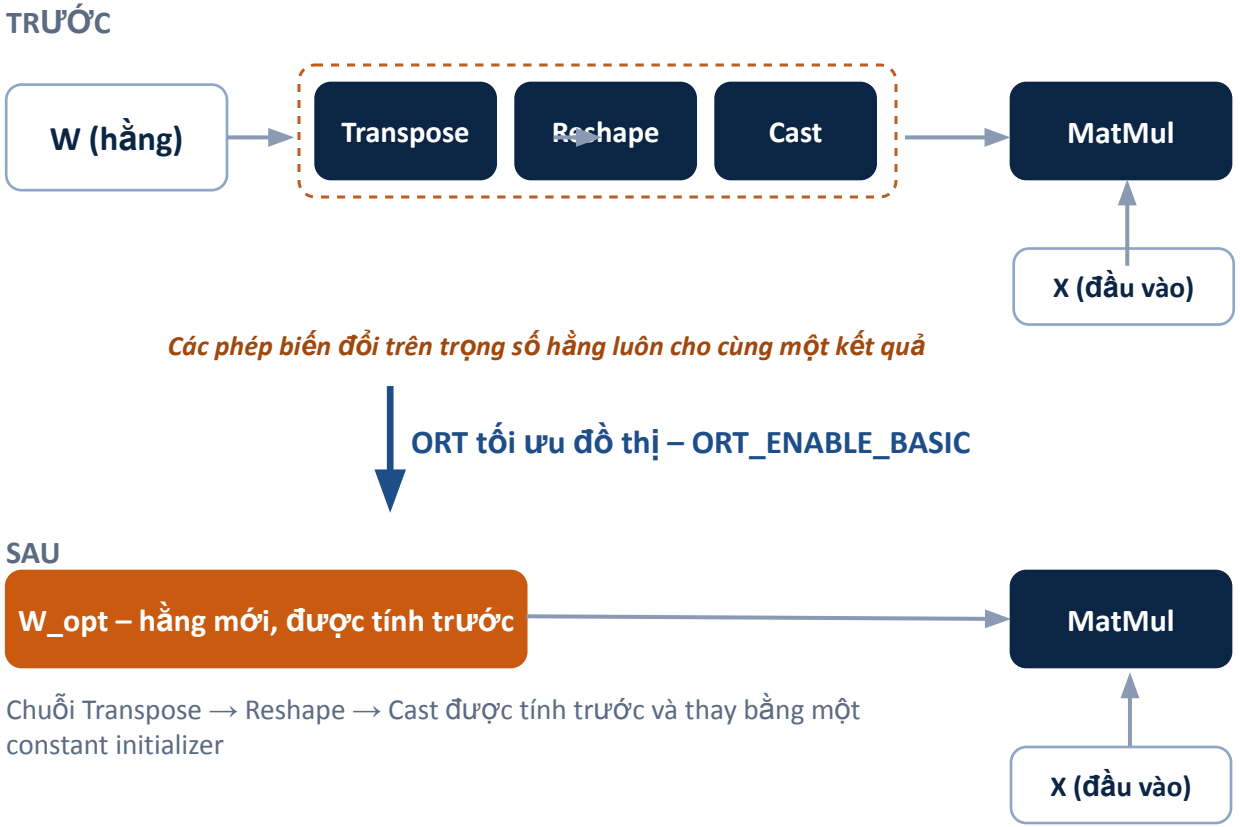
ORT_ENABLE_ALL

↓ bao gồm Basic + Extended

- Bao gồm Basic + Extended + Layout Optimization.
- Bổ sung NCHWc Optimizer cho các node phù hợp trên CPU.
- Chia chiều channel thành các block để hỗ trợ kernel vector hóa.
- Đây là mức graph optimization mặc định của ONNX Runtime.

Ví dụ: NCHW → NCHWc

Tối ưu mức Basic: Constant Folding



Cơ chế

- Chạy trước Graph Partitioning nên có thể áp dụng cho tất cả Execution Provider.
- Loại bỏ node hoặc phép tính dư thừa mà không thay đổi ngữ nghĩa của mô hình.
- Tính trước các phần đồ thị chỉ phụ thuộc vào hằng số.
- Thực hiện một số phép fusion cơ bản.

Các tối ưu khác ở mức Basic

Loại bỏ node dư thừa

Identity | Dropout | Unsqueeze | Slice

Fusion bảo toàn ngữ nghĩa

- Conv–Add Fusion
- Conv–Mul Fusion
- Conv–BatchNorm Fusion
- ReLU–Clip Fusion
- Reshape Fusion

Tối ưu mức Extended: Node Fusion

TRƯỚC — 3 kernel, 2 tensor trung gian



ORT tối ưu đồ thị – ORT_ENABLE_EXTENDED

SAU — 1 kernel hợp nhất (Streamlined Execution)



Đặc điểm của Node Fusion:

- Gộp chuỗi nhiều operator liên tiếp thành 1 kernel duy nhất
- Triệt tiêu hoàn toàn việc ghi/đọc các tensor trung gian ra RAM
- Tăng tốc độ tính toán nhờ tận dụng tối đa cache CPU/GPU

Cơ chế

- Bao gồm toàn bộ Basic Optimization.
- Extended Optimization chạy sau Graph Partitioning.
- Tìm các chuỗi operator đã được gán cho cùng Execution Provider.
- Thay chúng bằng fused operator hoặc kernel phù hợp.
- Giảm số node, tensor trung gian và overhead thực thi.

Hiệu quả của Node Fusion

- **Ít node hơn**
  Ít lần gọi kernel, ít chi phí điều phối
- **Không tensor trung gian**
  Bớt ghi/đọc bộ nhớ → giảm thắt cổ chai băng thông
- **Kernel hợp nhất**
  Tận dụng mã máy tối ưu sẵn cho CPU/CUDA

Cơ sở toán học của Conv–BatchNorm Fusion + Activation

1 **Convolution**

$$y = W * x + b$$

2 **BatchNorm ở chế độ inference** — μ, σ², γ và β là các tham số cố định sau huấn luyện; ε là hằng số nhỏ dùng để ổn định số học.

$$z = \gamma \frac{y - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

3 **Thay (1) vào (2) và nhóm các hạng tử**

$$z = \left(\frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} W \right) * x + \left(\frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} (b - \mu) + \beta \right)$$

- Các tham số được biến đổi theo từng output channel.
- Nếu Conv không có bias, xem b = 0.
- Phép biến đổi tương đương về mặt toán học ở chế độ inference.
- Kết quả số có thể sai khác rất nhỏ do tính toán dấu phẩy động.

ORT tính trước khi tối ưu đồ thị

$\alpha = \gamma / \sqrt{(\sigma^2 + \varepsilon)}$

$W' = \alpha W$

$b' = \alpha(b - \mu) + \beta$

Sau Basic Optimization:

$z = W' * x + b'$

BatchNorm được fold vào weight và bias của Conv.

Nếu áp dụng thêm Conv–Activation Fusion ở mức Extended trên CPU:

$out = ReLU(z) = max(0, z)$

→ Loại bỏ node BatchNorm và không cần tạo tensor trung gian riêng giữa Conv và BatchNorm.

Tối ưu mức Layout: NCHWc Optimizer

TRƯỚC — Layout NCHW

Tensor: [N, 64, H, W] — Dữ liệu được tổ chức theo Batch → Channel → Height → Width



64 channels liên tiếp

• Minh họa 64 kênh liên tiếp nhau trong bộ nhớ trước khi chuyển đổi layout.

ORT tối ưu layout — ORT_ENABLE_ALL

SAU — Layout NCHWc

Tensor: [N, 8, H, W, 8] **(Ví dụ minh họa với block c = 8)**



• Chiều Channel được chia thành các block phù hợp với kernel vector hóa của CPU.

• Ghi chú: Kích thước block c phụ thuộc phần cứng và kernel, không cố định luôn bằng 8 (ví dụ: AVX2 có thể dùng 8, AVX-512 dùng 16).

Đặc điểm của Layout Optimization:

- Thay đổi cách bố trí dữ liệu bên trong đồ thị.
- Có thể chèn các phép chuyển đổi layout tại ranh giới cần thiết.
- Không thay đổi ý nghĩa toán học của mô hình.
- Giao diện input/output của ứng dụng thường không cần thay đổi.

Cơ chế

- **ORT_ENABLE_ALL** bao gồm Basic, Extended và Layout Optimization.
- Layout Optimization chạy sau **Graph Partitioning**.
- Chỉ áp dụng cho các node phù hợp được gán cho **CPU Execution Provider**.
- NCHWc Optimizer chuyển dữ liệu từ NCHW sang dạng channel-blocked NCHWc.
- Mục tiêu là giúp kernel CPU khai thác vector hóa và truy cập bộ nhớ hiệu quả hơn.

Hiệu quả của NCHWc

• Vector hóa tốt hơn:
CPU có thể xử lý một nhóm channel trong cùng phép toán vector.

• Truy cập bộ nhớ hiệu quả hơn:
Dữ liệu được bố trí phù hợp hơn với các kernel CPU đã tối ưu.

• Giảm overhead chuyển đổi:
ORT cố gắng duy trì layout NCHWc giữa các operator tương thích.

** Mức tăng tốc phụ thuộc vào CPU, datatype, tensor shape, operator và chi phí chuyển đổi layout.*

Tối ưu quản lý bộ nhớ trong ONNX Runtime

Memory Arena

`enable_cpu_mem_arena`

Tái sử dụng các vùng nhớ đã cấp phát thay vì liên tục gọi malloc/free, giúp giảm overhead cấp phát và phân mảnh bộ nhớ. Arena có thể mở rộng theo nhu cầu, không nhất thiết cấp phát trước một khối cực lớn.

Memory Pattern

`enable_mem_pattern`

Ghi nhận mẫu cấp phát của các tensor trung gian trong lần chạy đầu. Khi input shape không đổi, ORT tái sử dụng mẫu này ở các lần chạy sau để giảm chi phí lập kế hoạch và cấp phát bộ nhớ.

Memory Reuse

`enable_mem_reuse`

Tái sử dụng vùng nhớ của các tensor trung gian có vòng đời không chồng lấp, nhờ đó giảm lượng bộ nhớ khi thực thi mô hình.

I/O Binding

`IOBinding (GPU/NPU)`

Cho phép gắn input và output với bộ nhớ trên thiết bị tăng tốc, giúp giảm các lần sao chép dữ liệu không cần thiết giữa CPU và GPU/NPU.

Thực nghiệm: mô hình phân loại Dodo (ResNet50)

Phân loại ảnh nhị phân

dodo → 0

no_dodo → 1

Mô hình

ResNet50 fine-tune bằng PyTorch; kế thừa checkpoint best_resnet50.pth — không huấn luyện lại

Tập kiểm thử

10 ảnh: 5 dodo + 5 no_dodo, không dùng trong huấn luyện

Vì sao ResNet50?

Nhiều khối Conv + BatchNorm + ReLU → nhiều cơ hội fusion

Tiền xử lý thống nhất

Áp dụng giống nhau cho PyTorch, ONNX Runtime và giao diện demo

Chuyển sang RGB

↓

Resize cạnh ngắn về 256 px

↓

Center crop 224 × 224

↓

ToTensor + chuẩn hóa ImageNet
mean (0.485, 0.456, 0.406) · std (0.229, 0.224, 0.225)

↓

Tensor [1, 3, 224, 224] · float32

Quy trình chuyển đổi và tối ưu ngoại tuyến

1 **Checkpoint best_resnet50.pth**

ResNet50 đã fine-tune, 2 lớp đầu ra (kế thừa bài tập trước)

2 **Khôi phục kiến trúc → nạp trọng số → eval()**

Chế độ suy luận: BatchNorm dùng thống kê cố định

3 **`torch.onnx.export`**

Đầu vào mẫu [1, 3, 224, 224] · đầu ra 2 logits · batch động

4 **ORT tối ưu ngoại tuyến (convert_to_onnx.py)**

Tối ưu cho CPUExecutionProvider, lưu thành 3 tệp:

`dodo_classifier_basic.onnx`	Basic
`dodo_classifier_extend.onnx`	Extended
`dodo_classifier.onnx`	All



Khi benchmark

Tắt tối ưu lúc nạp (disable) để giữ nguyên đồ thị đã lưu; ba tệp đều đã được tối ưu sẵn.



Kiểm chứng tính đúng

onnx.checker (cấu trúc) + so sánh logits PyTorch và ORT theo dung sai + kiểm tra nhãn argmax.



Demo Gradio (demo.py)

Chọn mô hình .pth hoặc .onnx → dự đoán dodo/no_dodo. Mã nguồn: github.com/sonnguyen112/AIoT

Thiết kế benchmark: cố định mọi yếu tố gây nhiễu

Thành phần	Thiết lập
Phần cứng	Intel Core i5-13420H · 12 CPU logic · Linux (WSL2), x86_64
Runtime	PyTorch CPU vs ORT CPUExecutionProvider · 1 luồng intra-op
Đầu vào	10 ảnh test · batch 1 · [1, 3, 224, 224] · float32
Warm-up	20 lượt × 10 ảnh = 200 lời gọi (không tính vào kết quả)
Đo lặp	1.000 lượt × 10 ảnh = 10.000 lời gọi / mô hình
Vùng đo	Chỉ lời gọi mô hình — không gồm đọc ảnh, tiền xử lý, nạp mô hình
Tối ưu khi nạp	Disable — dùng các đồ thị đã tối ưu ngoại tuyến
Cách ly	Mỗi mô hình chạy trong một process riêng

Phiên bản: Python 3.12.14 · PyTorch 2.14.0 · Torchvision 0.29.0 · ONNX Runtime 1.30.0 · ONNX 1.23.0 · NumPy 2.5.3 · Pillow 12.3.0

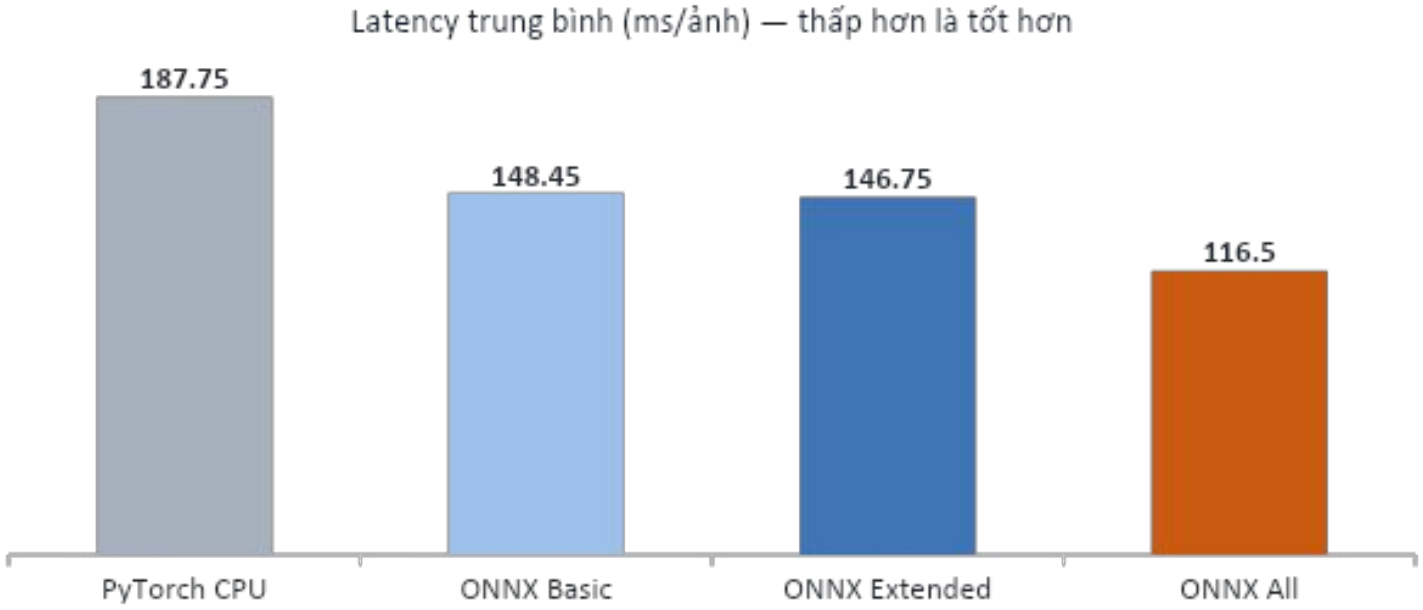
4 cấu hình được so sánh

- PyTorch CPU (mốc chuẩn)
- ONNX Basic
- ONNX Extended
- ONNX All

Chỉ số báo cáo

- Latency (ms/ảnh): Mean · Median · Std · P95
- Speedup = $T_{PyTorch} / T_{model}$
- Giảm latency = $(T_{PyTorch} - T_{model}) / T_{PyTorch}$
- Throughput ≈ 1.000 / latency (ms)

Kết quả: ONNX All nhanh hơn PyTorch CPU 1,61×



Cấu hình	Mean (ms)	Median	Std	P95	Speedup	ảnh/s
PyTorch CPU	187,75	180,96	34,58	254,67	1,00×	5,33
ONNX Basic	148,45	142,24	24,59	197,28	1,26×	6,74
ONNX Extended	146,75	140,19	24,99	196,75	1,28×	6,81
ONNX All	**116,50**	**111,12**	**20,75**	**156,98**	**1,61×**	**8,58**

1,61×

speedup của ONNX All so với PyTorch CPU

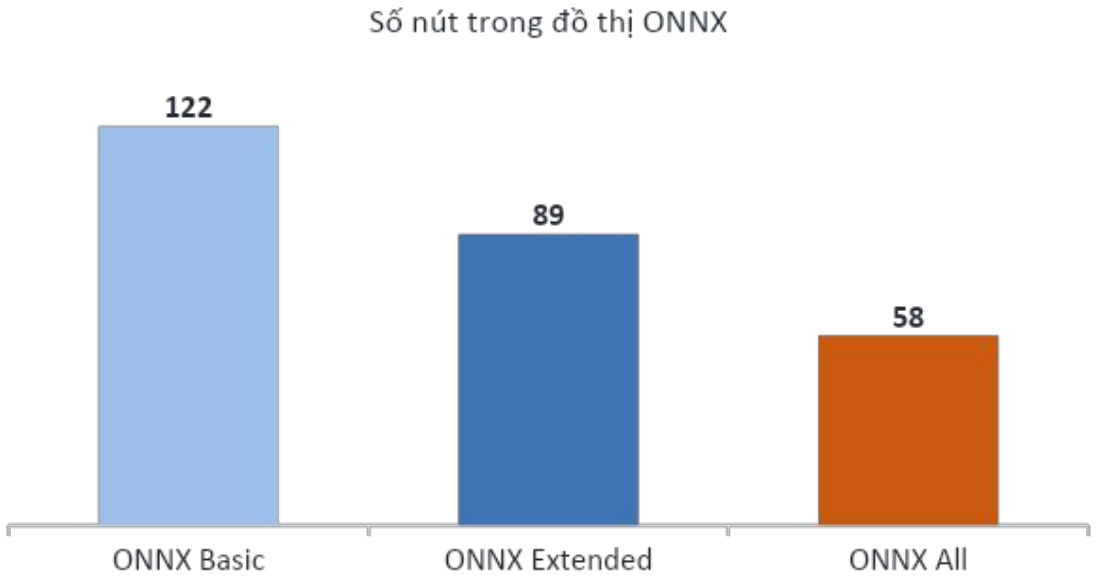
−37,95%

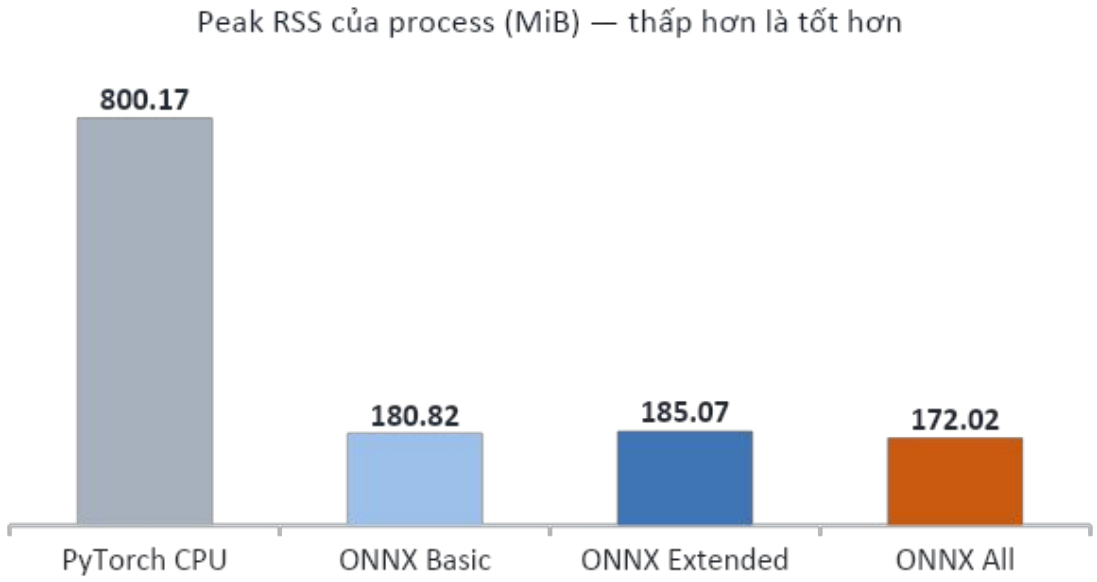
latency trung bình: 187,75 → 116,50 ms/ảnh

8,58 ảnh/s

throughput (PyTorch CPU: 5,33 ảnh/s, tăng ≈ 61%)

Kết quả: cấu trúc đồ thị, bộ nhớ và độ chính xác





Extended: 33 nút FusedConv (Conv–ReLU) · **All:** 56/58 nút layout NCHWc
Basic → All: −64 nút (−52,46%) — chênh lệch cấu trúc, không phải số lần fusion

RSS gồm cả runtime, thư viện và vùng cấp phát — không chỉ trọng số.

10/10
ảnh test đúng ở cả 4 cấu hình
5/5 dodo · 5/5 no_dodo

269,51 → 89,61 MiB
tệp .pth (gồm cả trạng thái optimizer) → tệp .onnx

≈ 100% một core
tương đương ≈ 8,39% toàn máy 12 CPU logic — hai cách chuẩn hóa khác nhau

Kết luận

01

Biểu diễn chung

ONNX đưa mô hình về một biểu diễn chung, tách suy luận khỏi framework huấn luyện — trong phạm vi toán tử và backend được hỗ trợ.

02

Tối ưu hóa đồ thị

ONNX Runtime tối ưu đồ thị (Constant Folding, Node Fusion, Layout), quản lý bộ nhớ và phân phối tới Execution Provider phù hợp phần cứng.

03

Thực nghiệm Dodo

1,61× Nhanh hơn

ONNX All đạt **116,50 ms/ảnh** và **8,58 ảnh/s** — nhanh hơn PyTorch CPU 1,61×, vẫn đúng **10/10 ảnh test**.

Tài liệu tham khảo

[1] ONNX, “About ONNX” — onnx.ai/about

[2] ONNX, “Overview – Why a common IR?” — onnx.ai/onnx/repo-docs/Overview.html

[3] ONNX, “ONNX Concepts” — onnx.ai/onnx/intro/concepts.html

[4] ONNX, “ONNX IR Specification” — onnx.ai/onnx/repo-docs/IR.html

[5] ONNX, kho mã nguồn chính thức — github.com/onnx/onnx

[6] ONNX Runtime, “Graph optimizations” — onnxruntime.ai/docs/performance/model-optimizations/graph-optimizations.html

[7] ONNX Runtime, “I/O Binding” — onnxruntime.ai/docs/performance/tune-performance/iobinding.html

[8] ONNX Runtime, “Reduce memory consumption” — onnxruntime.ai/docs/performance/tune-performance/memory.html

[9] ONNX Runtime, “Troubleshooting performance issues” — onnxruntime.ai/docs/performance/tune-performance/troubleshooting.html

[10] ONNX Runtime, “Custom build” (minimal build, định dạng .ort) — onnxruntime.ai/docs/build/custom.html

[11] ONNX Runtime, “Python API Summary” — onnxruntime.ai/docs/api/python/api_summary.html

[12] PyTorch, “Export a PyTorch model to ONNX” — docs.pytorch.org/tutorials/beginner/onnx/export_simple_model_to_onnx_tutorial.html

[13] Facebook & Microsoft (09/2017), “… new open ecosystem for interchangeable AI frameworks” — research.facebook.com/blog/2017/9/

Phân công nhiệm vụ & AI Disclosure

Bảng phân công nhiệm vụ thành viên

#	Tên thành viên	Nhiệm vụ
01	**Trần Ngọc Phương Dung**	Phân công nhiệm vụ và soạn thảo nội dung về tối ưu hóa đồ thị của ORT và tối ưu hóa bộ nhớ đệm.
02	**Cao Anh Tuấn**	Xây dựng kịch bản demo và thực hiện Benchmark đo đạc tốc độ suy luận, bộ nhớ và lập báo cáo so sánh chi tiết.
03	**Mai Nguyễn Hữu Hùng**	Soạn thảo nội dung về ONNX và ONNX runtime
04	**Nguyễn Hồ Trường Sơn**	Thực nghiệm huấn luyện và chuẩn hóa dữ liệu đầu vào. Tối ưu hóa đồ thị bằng ONNX Runtime
05	**Nguyễn Duy Thanh**	Tổng hợp nội dung, biên tập nội dung Slide thuyết trình và hoàn thiện tài liệu.

Tuyên bố sử dụng AI (AI Disclosure)

- **Codex:** Hỗ trợ lập trình & Gỡ lỗi (Debugging), tra cứu cú pháp thư viện, gợi ý cấu trúc lệnh, giải thích thông báo lỗi.
- **Gemini / Perplexity:** Tra cứu & Tổng hợp tài liệu, tóm tắt khái niệm kỹ thuật phức tạp, giải thích công thức toán học, tìm kiếm tài liệu tham khảo.
- **Claude:** Hỗ trợ biên tập Slide, báo cáo & dựng template; tối ưu hóa câu từ, sửa lỗi chính tả/ngữ pháp và định dạng văn bản mạch lạc.

Mức độ đóng góp của sinh viên:

Các thành viên trong nhóm đã trực tiếp kiểm tra, xác thực mã nguồn, đo đạc dữ liệu thực tế và chịu trách nhiệm 100% về tính chính xác của toàn bộ nội dung báo cáo.

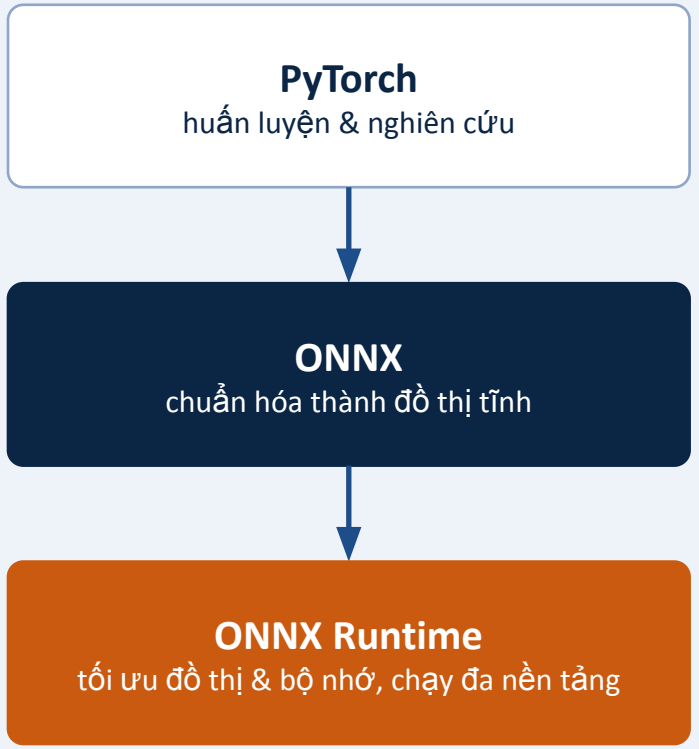
Cảm ơn thầy và các bạn đã lắng nghe!

Hỏi & Đáp

Nhóm 06 · Học phần AIoT – Trí tuệ nhân tạo vạn vật kết nối

github.com/sonnguyen112/AIoT

TÓM TẮT



1,61× nhanh hơn · **10/10** ảnh đúng